Контрольная работа №2
по дисциплине «ПО объектов наземной инфраструктуры»

Тема: Абстрактные и виртуальные классы в архитектуре БПЛА
Специальность: 27.03.05 Инноватика
Продолжительность: 4 академических часа (180 минут)
Форма проведения: письменная работа с выполнением программных заданий в Qt Creator
Разрешённые материалы: лекция №3, документация Qt, личные конспекты

Цель контрольной работы

Проверить умение самостоятельно, без пошаговых указаний:

- использовать наследование, абстрактные классы и виртуальные методы;
- применять полиморфизм для проектирования заменяемых модулей беспилотной системы;
- соблюдать инкапсуляцию: скрывать данные, предоставлять доступ через методы;
- обрабатывать ошибочные ситуации;
- оформлять проект в Qt Creator с разделением на .h и .cpp.

Общие требования ко всем вариантам

Создайте проект:

```
Control_Work_02_Polymorphism
```

Рекомендуемая структура:

```
Control_Work_02_Polymorphism/
├── main.cpp
├── uav.h        uav.cpp
├── sensor.h     sensor.cpp
├── navigation.h   navigation.cpp
└── controller.h   controller.cpp
```

Обязательные требования к коду:

- базовые классы — абстрактные, с виртуальным деструктором virtual ~Имя() = default;;
- каждый переопределённый метод сопровождается override;
- поля классов объявляются в private (или protected, если они нужны наследникам), доступ — только через методы;
- универсальные функции работают с абстрактными ссылками/указателями и **не содержат проверок типа через** if/dynamic_cast;
- вывод — через qDebug();
- проверяется хотя бы одна ошибочная ситуация (нулевые указатели, неготовый сенсор, невалидная навигация).

Задание 1. Виртуальные классы летательных аппаратов (1 балл)

Создайте абстрактный базовый класс:

```
class UAV {
public:
   virtual void takeoff() = 0;
   virtual void land() = 0;
   virtual QString type() const = 0;
   virtual ~UAV() = default;
};
```

Вариант 1

Создайте классы Quadcopter и FixedWingUAV. Ожидаемый вывод:

```
Quadcopter: vertical takeoff
Quadcopter: vertical landing
Fixed-wing UAV: runway takeoff
Fixed-wing UAV: gliding landing
```

Реализуйте функцию:

```
void run_flight_test(UAV& vehicle);
```

Функция выводит тип аппарата, выполняет взлёт и посадку, выводит сообщение о завершении испытания — **не зная фактического типа аппарата**.

Вариант 2

Создайте классы Quadcopter, Hexacopter, Octocopter.

Добавьте в базовый класс чисто виртуальный метод:

```
virtual int motor_count() const = 0;
```

Ожидаемый вывод:

```
Quadcopter: 4 motors
Hexacopter: 6 motors
Octocopter: 8 motors
```

Реализуйте функцию print_vehicle_info(const UAV& vehicle), выводящую тип, число моторов, тип взлёта и тип посадки.

Вариант 3

Создайте классы SurveyUAV, CargoUAV, ReconnaissanceUAV.
Добавьте метод:

```
virtual QString mission_type() const = 0;
```

Ожидаемый вывод:

```
Survey UAV: terrain mapping
Cargo UAV: payload transportation
Reconnaissance UAV: observation and monitoring
```

Реализуйте функцию execute_test_mission(UAV& vehicle), работающую с любым производным типом без проверки типа через if.

Задание 2. Абстрактный класс сенсора (1 балл)

Создайте абстрактный базовый класс:

```
class Sensor {
public:
   virtual double read() = 0;
   virtual bool is_ready() const = 0;
   virtual QString name() const = 0;
   virtual ~Sensor() = default;
};
```

Соберите сенсоры в std::vector<Sensor*> и выполните полиморфный обход.

Вариант 1 — сенсоры полётного контроллера

Классы Gyroscope, Accelerometer, Barometer.
Контрольные значения:

```
Gyroscope: 1.25 rad/s
Accelerometer: 9.81 m/s2
Barometer: 1012.8 hPa
```

Реализуйте функцию check_sensors(const std::vector<Sensor*>& sensors), которая выводит имя и состояние каждого сенсора и формирует общий статус:

```
Checking sensors...
Gyroscope: Ready, value=1.25
Accelerometer: Ready, value=9.81
Barometer: Ready, value=1012.8
All sensors are ready
```

Обязательно проверьте сценарий неготового сенсора: измените состояние одного из них (через сеттер) и убедитесь, что общий статус меняется на «sensor failure».

Вариант 2 — навигационные сенсоры

Классы GPS, GLONASS, VisualOdometry.
Добавьте в интерфейс метод:

```
virtual double accuracy() const = 0;
```

Ожидаемый вывод:

```
GPS: satellites=10, accuracy=1.5 m
GLONASS: satellites=8, accuracy=2.0 m
Visual odometry: accuracy=3.2 m
```

Реализуйте функцию выбора лучшего источника:

```
Sensor* choose_best_sensor(const std::vector<Sensor*>& sensors);
```

Критерии: сенсор должен быть готов; выбирается сенсор с минимальной ошибкой. Обработайте случай, когда ни один сенсор не готов (возврат nullptr и сообщение об ошибке).

Вариант 3 — сенсоры окружающей обстановки

Классы Lidar, Radar, Camera. Семантика read():

Класс	Что возвращает read()
Lidar	расстояние до ближайшего препятствия, м
Radar	расстояние до объекта, м
Camera	число объектов в кадре

Ожидаемый вывод:

```
Lidar: nearest obstacle=12.5 m
Radar: nearest object=18.2 m
Camera: detected objects=3
```

Реализуйте функцию:

```
bool obstacle_is_near(Sensor& sensor, double threshold);
```

Проверьте её на порогах 10 м и 20 м — результат должен различаться корректно.

Задание 3. Абстрактная система навигации (1 балл)

Базовый интерфейс для вариантов 1 и 2:

```
struct Position {
   double latitude;
   double longitude;
   double altitude;
};
class NavigationSystem {
public:
   virtual Position get_position() const = 0;
   virtual bool is_valid() const = 0;
   virtual QString system_name() const = 0;
   virtual ~NavigationSystem() = default;
};
```

Вариант 1 — GPS и визуальная навигация

Классы GPSNavigation (поля: position, satellites, valid; метод is_valid() возвращает satellites >= 4 && valid) и VisualNavigation (поле target_tracked; is_valid() возвращает его значение).

Реализуйте функцию print_navigation_status(const NavigationSystem& navigation):

```
Navigation system: GPS
Position: 55.75, 37.61, 150
Status: Valid
```

Проверьте оба состояния: валидную и невалидную навигацию.

Вариант 2 — отказоустойчивая навигация

Классы GPSNavigation, InertialNavigation, VisualNavigation.
Создайте класс NavigationManager:

```
class NavigationManager {
private:
   NavigationSystem* active_system;   // инкапсуляция: поле закрыто
public:
   void set_system(NavigationSystem* system);
   bool is_available() const;
   Position get_position() const;
   void print_status() const;
};
```

Алгоритм переключения: GPS → инерциальная → визуальная; если все недоступны — критическая ошибка; активной становится первая доступная система. Имитируйте отказ: gps.set_valid(false); — после этого программа должна автоматически переключиться на резервную систему:

```
GPS: invalid
INS: valid
Active navigation system: INS
```

Вариант 3 — абстрактные контроллеры полёта

Абстрактный класс:

```
class FlightController {
public:
   virtual double calculate_control(double error, double dt) = 0;
```

```
    virtual QString controller_name() const = 0;
    virtual ~FlightController() = default;
};
```

Создайте PIDController, SimpleController, FuzzyController.
Формула ПИД-регулятора (состояние — private):

```
integral += error * dt;
derivative = (error - previous_error) / dt;
output = kp * error + ki * integral + kd * derivative;
previous_error = error;
```

Простой регулятор: output = coefficient * error;
Реализуйте функцию test_controller(FlightController& controller, double error, double dt).

Пример:

```
Controller: PID
Error: 2.5
Control output: 1.87
```

Протестируйте все три контроллера через один и тот же вызов — это и есть полиморфный сценарий.

Интеграционное задание. Класс Autopilot (1 балл)

Выполняется во всех вариантах. Объедините результаты трёх заданий в упрощённую систему автопилота:

```
class Autopilot {
private:
    UAV* vehicle;
    std::vector<Sensor*> sensors;
    NavigationSystem* navigation;
    FlightController* controller;
public:
    Autopilot(UAV* vehicle,
              const std::vector<Sensor*>& sensors,
              NavigationSystem* navigation,
              FlightController* controller);
    void run_cycle();
};
```

Алгоритм run_cycle():

1. Проверить наличие БПЛА (vehicle == nullptr → Error: vehicle is not set, выход).
2. Проверить сенсоры: для каждого — указатель, готовность, значение. Если хотя бы один критически важный сенсор не готов — прервать цикл.
3. Проверить навигацию: указатель не нулевой, is_valid() возвращает true.
4. Получить позицию через navigation->get_position().
5. Вычислить ошибку по высоте: error = target_altitude - position.altitude при target_altitude = 200.0.
6. Вычислить управление: controller->calculate_control(error, 0.1).
7. Вывести результат.

Эталонный вывод при штатной работе:

```
Autopilot cycle started
All sensors are ready
Navigation: GPS
Current altitude: 150 m
Target altitude: 200 m
Altitude error: 50 m
Controller output: 25
Autopilot cycle completed
```

Обязательные тестовые сценарии

№	Сценарий	Ожидаемый результат
1	Все сенсоры готовы, GPS валиден, высота 150 м	Полный цикл, ошибка 50 м, вывод управления
2	Сенсор не готов	Autopilot cycle aborted / Reason: sensor failure
3	Навигация невалидна	Navigation data invalid
4	controller == nullptr	Error: flight controller is not set
5	Некорректная координата (отрицательная высота)	Error: invalid navigation position